

Automated process for analyzing 2D CAD floor plan drawings and generating FloorspaceJS-compatible space objects for building energy simulations

Wei-Jen Chen^{a,*}, Min-Cheng Tu^b

^a Undergraduate Program of Vehicle and Energy Engineering, National Taiwan Normal University, Taipei City 106, Taiwan

^b Department of Natural Sciences, College of Coastal Georgia, Brunswick, GA 31520, USA

ARTICLE INFO

Keywords:

Building energy simulation
Automation
CAD drawing analysis
2D Space Generation

ABSTRACT

Building energy simulation has been widely utilized to support the design decision-making process for new constructions and to establish baselines for energy retrofitting of existing buildings. However, the requirement for well-trained engineers to spend substantial time completing the work has somewhat hindered its development. This paper attempts to extract geometric data from existing 2D floor plan drawings and then proposes an algorithm to identify all faces enclosed by simple closed curves on the drawings. These faces are subsequently converted into the FloorspaceJS format files supported by the OpenStudio Application, which can be directly imported for further processing. The results indicate that the time complexity of this process, represented using Big O notation to describe the upper bound of algorithmic runtime in computational complexity analysis, is $O(NM)$, where N represents the total number of vertices and M represents the total number of line segments in the relevant layer of the drawing. Given that N and M are typically within manageable limits, the algorithm is expected to complete its task in seconds. As a result, this approach is well-suited to accelerating building energy simulation, thereby saving engineers valuable time.

1. Introduction

From the early architectural concept design [1] to the later stages of detailed space-by-space HVAC (Heating, Ventilation, and Air Conditioning) calculations [2], building simulation has become an effective tool for practical decision-making [3]. In order to meet certain government tax credit requirements, such as the Internal Revenue Service's energy-efficient commercial building property deduction in the United States, current industry standard practices tend to use ASHRAE (American Society of Heating, Refrigerating and Air-Conditioning Engineers) standard 140 certified building simulation software, which include Design Builder, EnergyPlus, eQuest, Hourly Analysis Program (HAP), IES Virtual Environment, Trace 700, and Trnsys [4]. Among these, EnergyPlus has been widely recognized as one of the most extensively used building energy simulation engines, supported by studies such as Shabunko et al. [5].

Sponsored by the U.S. Department of Energy (DOE) and developed primarily by the National Renewable Energy Laboratory (NREL) and Lawrence Berkeley National Laboratory (LBNL), EnergyPlus is a free,

open-source project. Compared to its predecessor software, DOE-2, and BLAST (Building Loads Analysis and System Thermodynamics), EnergyPlus inherits some advantages and overcomes some drawbacks, ultimately providing more accurate and flexible building energy consumption simulation capabilities. This makes EnergyPlus the most popular software in the field of building energy simulation.

According to the Department of Energy, it's more accurate to describe EnergyPlus as a BEM (Building Energy Modeling) engine rather than software. In fact, EnergyPlus does serve as the underlying computational core for some of the aforementioned building simulation software (e.g., Design Builder). Regarding a car engine, while it theoretically allows a person to embark on a journey by itself, it's actually difficult to do so without other essential components. Similarly, the main issue with the BEM engine EnergyPlus is the lack of a graphical interface for constructing geometry. Although one with a profound understanding of EnergyPlus can theoretically "construct" the geometric structure of an entire building using a plain text editor, this is practically next to impossible for large projects. Due to this limitation, EnergyPlus often needs to be paired with other tools for geometry. Among these

* Corresponding author.

E-mail address: kylechen@ntnu.edu.tw (W.-J. Chen).

<https://doi.org/10.1016/j.rineng.2025.105575>

Received 5 April 2025; Received in revised form 18 May 2025; Accepted 30 May 2025

Available online 31 May 2025

2590-1230/© 2025 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

auxiliary tools, OpenStudio has been extensively adopted in the industry as a major interface for EnergyPlus, as evidenced by its official endorsement by the U.S. Department of Energy and its widespread integration into both academic research and professional practice. Originally launched in 2006 as a SketchUp plugin for EnergyPlus geometry generation, it has evolved into its current state, including the OpenStudio Application with a user interface and the highly programmable OpenStudio SDK [6].

Today, OpenStudio has optimized many processes in building simulation. It offers construction sets for quickly handling building materials, space types as space templates for rapid space assignment and schedule sets for defining hourly-based variations. Furthermore, it even provides predefined HVAC system templates. However, one thing that remains challenging and is likely to always be so is the geometry generation of the building. Because building shapes vary greatly, each building simulation project requires a specific geometric structure to be made—a very labor-intensive task. Actually, geometry modeling has been identified as one of the most time-consuming stages in building energy simulation workflows, particularly for buildings with complex structures, according to a study focusing on modeling bottlenecks [3].

Regarding the geometry generation, if the building to be simulated is aged and no architectural drawings are available, it will require further on-site surveys and measurements to obtain accurate building dimensions and details. If relevant documents such as CAD (Computer-Aided Design) drawings exist, the task becomes somewhat easier. However, regardless of the data used, once the detailed information is available, engineers must manually construct the geometry of the building by inputting the detailed dimensions one by one. To speed up the process, engineers sometimes create drawings with a scale, import them, and then appropriately resize them to use as background patterns for tracing contours. Regardless of the approach employed, as the building size increases, these tasks require experienced engineers to work day and night for tens of hours to complete them, which is a significant obstacle to achieving rapid building simulation.

To address the aforementioned problems, a new generation of tools, known as Building Information Modeling (BIM) software, has been released. Research indicates that BIM tools can significantly reduce the time required to create building models [7]. However, for BEM applications, accurate simulation requires not only geometric detail but also the correct representation of spatial relationships between thermal zones. One critical concept in this regard is the second-level space boundary (2LSB), which defines paired surfaces that explicitly capture the adjacency and thermal interactions between spaces. While first-level space boundaries represent basic geometric partitions, 2LSBs ensure that no surface is double-counted and that simulation boundary conditions are applied correctly. Given these requirements, Bazjanac [8] highlighted the role of 2LSBs in energy simulation workflows, and several algorithms have since been developed to automatically generate them from IFC-based BIM models. These include dimensional reduction methods [9], topology-driven IFC extraction [10], automated thermal zoning [11], and surface partitioning algorithms [12]. These studies form the foundation for automated, simulation-ready model generation.

Although our method currently operates in 2D using FloorspaceJS, it shares fundamental spatial logic with 2LSB-based approaches. For instance, edge-based zone partitioning in floor plans indirectly reflects the geometric reasoning required in 2LSB derivation. Incorporating the insights from prior 2LSB research thus strengthens the theoretical basis of our approach and helps extend its potential to full 3D support in the future. Ultimately, building on 2LSB principles supports the broader goal of automating the CAD-to-BEM transformation, making it faster and more accessible to generate valid simulation models from early-stage design geometry.

In parallel, prior research has also addressed the challenge of automatically extracting spatial topology and room connectivity from 2D architectural data. Domínguez et al. [13] proposed a semiautomated method to detect floor plan topology from CAD-based architectural

drawings, demonstrating an early attempt to bridge raw drawing data and room structure reconstruction. Their approach introduces a data structure known as the Wall Adjacency Graph (WAG), which encodes the spatial relationships between wall segments to infer room boundaries. Lewis and Séquin [14] contributed a foundational approach that converts 2D plans into 3D building models, emphasizing geometric consistency in plan-to-model transitions. More recently, Lilis et al. [15] introduced an automated method to enrich IFC models with spatial geometries to support energy simulations, while Lin et al. [16] leveraged BIM and 3D-GIS techniques to automate indoor space topology generation for navigation purposes.

In a related development, Fang et al. [17] proposed an automated "Scan-to-BIM-to-Sim" workflow that converts bridge point cloud data into BIM models and connects them to finite element simulation (FEM) tools. Their approach not only handles low-quality point clouds but also enables bidirectional data flow and real-time synchronization between BIM and simulation models. Although this application is in the domain of bridge engineering, the underlying concept of automating model generation and simulation integration aligns closely with the goal of this study—namely, to automate the creation of energy simulation-ready geometry from 2D CAD drawings. Complementing this line of research, Altaf et al. [18] developed an automated LCCA (Life Cycle Cost Analysis) tool that integrates directly into the BIM environment. Their work focuses on automating the economic evaluation of building projects by extracting component information from BIM models and performing LCCA calculations within Revit. While their primary objective is financial analysis rather than geometric modeling, their study similarly demonstrates the power of embedded BIM automation tools to streamline simulation-related workflows and support early-stage design decisions.

Despite these advancements, however, the widespread adoption of BIM tools remains limited in practice. While modern platforms such as Autodesk Revit now support direct export to gbXML—a geometric format compatible with simulation tools like OpenStudio—this technological capability has not yet translated into ubiquitous use. Unfortunately, the shift from traditional CAD software (e.g., AutoCAD) to BIM software (e.g., Revit) is progressing slowly and incrementally, rather than through a rapid industry-wide transformation. Based on the authors' observations, many new building designs are still drafted using CAD rather than BIM, not to mention legacy buildings where BIM was never incorporated. Although it is technically possible to re-interpret and re-draw CAD files using BIM software, this process remains labor-intensive and contradicts the goal of achieving rapid and efficient building simulation.

Therefore, automating the identification of CAD drawings is a crucial task. Earlier work by Lebeque and Aggarwal [19] explored a more novel direction by using a mobile robot to capture scene images, reconstructing 3D geometry through structure-from-motion techniques, and then converting these into CAD models in DXF format. This vision-based approach minimizes human input and demonstrates an alternative path toward automated model generation from physical environments.

Recent advances in automated CAD file generation have contributed to reducing the manual effort required in architectural modeling and building simulation. For instance, Bortoluzzi et al. [20] developed a semi-automated pipeline that converts 2D CAD drawings into BIM models using Dynamo scripts within Autodesk Revit. Their approach helps automate Revit model creation for existing buildings, significantly reducing the modeling burden for retrofit projects.

As automation in architectural modeling progresses, scholars have addressed the interoperability and translation challenges between CAD formats and simulation-ready models. Cicirello and Regli [21] proposed a flexible framework for automated classification of engineering file formats, independent of specific modeling tools. Their approach improves the recognition and interpretation of CAD/CAM file types, enabling more seamless integration across platforms and facilitating digital workflows for downstream applications such as simulation and

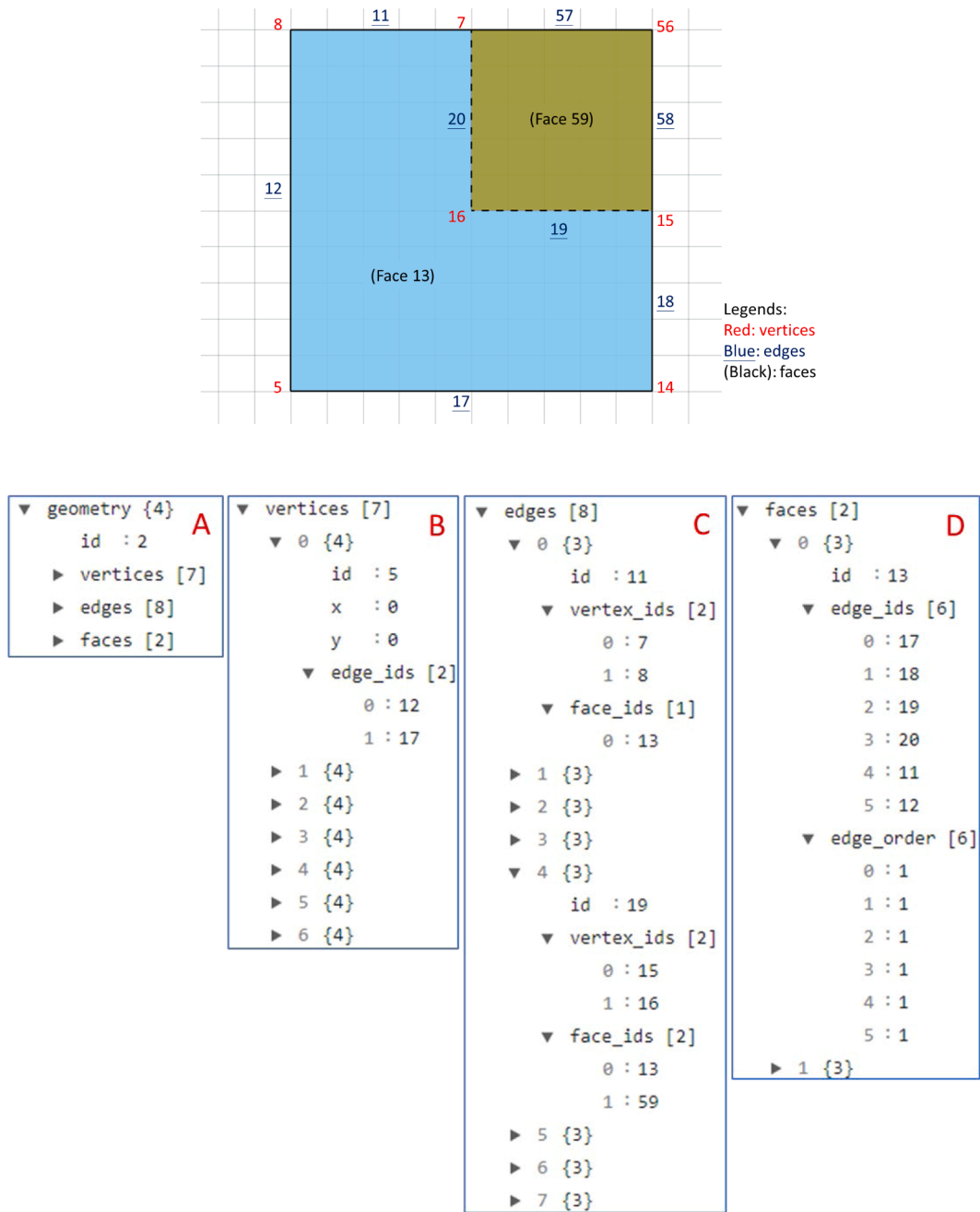


Fig. 1. Example floor plan and corresponding FloorspaceJS geometry object structure. (visualized by JSON Viewer).

manufacturing.

To achieve automated identification, it is essential to have a comprehensive understanding of CAD drawing data formats. In the MEP (Mechanical, Electrical, and Plumbing) field, the DWG format has become the most widely used CAD drawing standard since its introduction alongside AutoCAD in 1982 [22]. Consequently, many teams have conducted further research on DWG, achieving significant progress. For example, Zhao et al. [23] analyzed the internal format of DWG files, covering basic and extended data types. Zhang and Shi [24] provided a detailed analysis of the AutoCAD drawing file format and utilized Visual C++ and ODBC interfaces to establish a graphics database, allowing users to easily browse, modify, and query drawings, supporting preview, zoom, and pan functionalities.

In addition to individual teams achieving similar goals through different technical paths, some teams have packaged their achievements into widely adopted APIs, leading to more comprehensive applications.

For instance, DWGdirect (now known as ODA Drawings SDK), developed by the Open Design Alliance, exemplifies such a case. ODA Drawings SDK can convert DWG files into other formats such as DXF and DGN. The SDK provides extensive graphical object manipulation capabilities, including the creation, editing, and deletion of lines, circles, text, and blocks. Furthermore, it supports layer management, employs event handling mechanisms to monitor and respond to graphical operations, offers file encryption and decryption, data extraction, and report generation features, and allows developers to extend and customize through a rich API. These functionalities make ODA Drawings SDK a tool of choice for many researchers [25,26].

Through the efforts of the aforementioned research groups, reading or even manipulating DWG files is no longer a difficult task, and there are many ready-made toolsets available on various platforms, such as the ODA Drawings SDK for C++ [25], pyAutocad for Python [27], and the AutoCAD .NET API for .NET platforms (C#, VB.net) [28]. However,

there is still a gap between being able to read DWG files and automatically generating geometric structure information for OpenStudio. The main reason for this is that the information needed for the latter does not actually exist in the original DWG files; it needs to be derived by mutating the original data.

Currently, there are few studies on this aspect of automation, so the main goal of this paper is to fill this gap. The study aims to explore how to extract information from architectural firm's 2D CAD floor plan drawings and automatically convert it into a format supported by OpenStudio, thus, eliminating the tedious steps for engineers to manually input dimensions or trace contours based on the drawings, thereby expediting building simulations.

The remainder of this paper is organized as follows. Section 2 presents the proposed methodology, including the structure of FloorspaceJS, the preprocessing workflow for CAD drawings, and the path-tracing algorithm for face detection. Section 3 provides results from both controlled examples and real-world CAD drawings, followed by a discussion on performance and limitations. Section 4 outlines the key assumptions and constraints of the approach, while Section 5 concludes the paper and suggests directions for future work.

2. Methodology

2.1. Overview

This study aims to automate the generation of geometric models for building energy simulation by converting 2D CAD floor plans into FloorspaceJS-compatible formats. The methodology is designed to address three core questions: (1) What geometric data is required by OpenStudio for energy modeling? (2) What information can be extracted from 2D floor plans? (3) How can this information be algorithmically transformed into simulation-ready geometry without manual intervention?

The proposed pipeline consists of three stages: input data preprocessing, face generation, and JSON conversion. Each stage will be described in the following subsections.

2.2. Requirements of FloorspaceJS geometry objects

In the current release of the OpenStudio Application, users may construct 3D geometric models via four primary pathways: (1) utilizing the built-in FloorspaceJS editor for 2D floor plan sketching and meta-data input; (2) importing geometry from gbXML files; (3) importing pre-constructed IDF files; and (4) importing existing OSM files. Among these, the third and fourth approaches presuppose the availability of pre-modeled 3D geometry and are therefore excluded from the scope of this study.

The focus of this research lies in the first two methods, which differ primarily in data format: FloorspaceJS relies on a JSON (JavaScript Object Notation) schema, while gbXML adopts XML (Extensible Markup Language). Given the structural similarities in terms of the required geometric data, either format can be programmatically generated from appropriately processed 2D CAD inputs. For simplicity and clarity in algorithm development, this study adopts FloorspaceJS as the target format for conversion.

FloorspaceJS, a 2D geometry editor within the OpenStudio Application, organizes spatial geometry using vertex, edge, and face arrays. Each floor (termed a "story") contains space objects referencing these geometric primitives. To generate valid faces, each must be composed of a sequence of counterclockwise edges enclosing a simple closed loop. Thus, automatic generation requires not only accurate edge identification but also the correct logical grouping of edges into Faces.

2.3. Representation framework of FloorspaceJS geometry objects

The FloorspaceJS schema is adopted as the output target format for

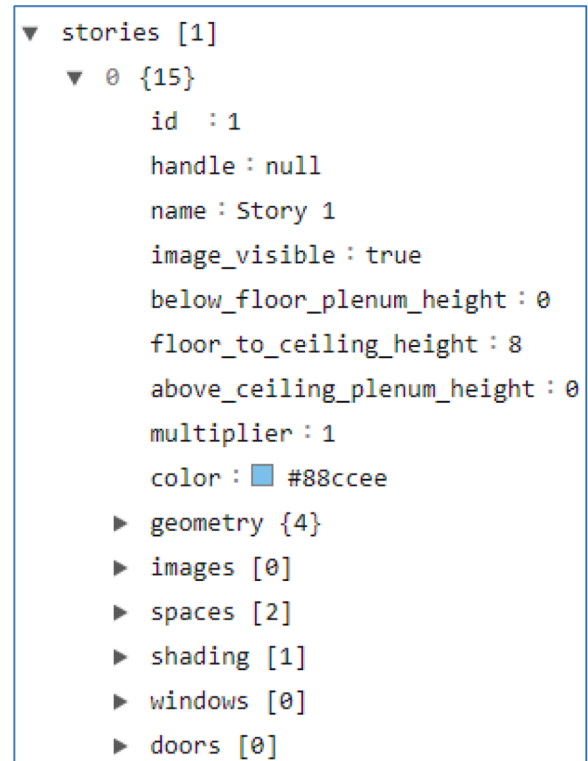


Fig. 2. JSON structure of a story object in FloorspaceJS (visualized by JSON Viewer).

automatically generating geometric data compatible with the OpenStudio Application. FloorspaceJS expresses building geometry through a structured JSON format, encompassing arrays of vertices, edges, and faces organized by story level. This data structure provides a machine-readable yet flexible representation of spatial configurations, making it well-suited for procedural geometry generation.

Fig. 1. illustrates a sample 2D floor plan within FloorspaceJS alongside its corresponding geometric data representation. The geometry object consists of four key components: an object identifier (id), a vertex list, an edge list, and a face list. Each geometric primitive follows a defined structure: vertices are defined by their coordinates and the edges to which they belong; edges, in turn, specify their endpoint vertices and associated faces; and faces are enclosed by an ordered sequence of edges, which must be specified in a counterclockwise direction to ensure validity in simulation environments.

As shown in the left portion of Fig. 1 (column A), each geometry object begins with a unique id. In the vertex array (column B), each vertex entry contains an identifier, x and y coordinates, and a list of associated edge identifiers. For example, vertex id = 5 has coordinates (0, 0) and is shared by edges 12 and 17. Edges (column C) are defined by their endpoint vertex IDs and face associations. Notably, edges may be shared across multiple faces, as in the case of edge 19, which is part of both face 13 and face 59. Finally, face objects (column D) reference a sequence of edge IDs and include an edge_order parameter, which is currently fixed to 1 in practice.

Fig. 2 further depicts the structure of a story object in FloorspaceJS. Each floor is defined with a unique id, a name, and optional metadata such as floor-to-ceiling height, plenum heights, and color coding. Additional arrays for windows, doors, and shading objects are also supported but are not utilized in the present implementation. The images array can optionally include background drawings to facilitate manual sketching when desired. Of particular interest is the organization of space and geometry objects within each story, which are hierarchically structured to ensure consistent floor-level representation.

▼ spaces [2]

▼ 0 {19}
id : 3
handle : null
name : Space 1 - 1
face_id : 13
building_unit_id : null
thermal_zone_id : null
space_type_id : null
construction_set_id : null
pitched_roof_id : null
▶ daylighting_controls [0]
below_floor_plenum_height : null
floor_to_ceiling_height : null
above_ceiling_plenum_height : null
floor_offset : null
open_to_below : null
building_type_id : null
template : null
color : ■ #88ccee
type : space
▶ 1 {19}

▼ spaces [2]
▶ 0 {19}
▼ 1 {19}
id : 39
handle : null
name : Space 1 - 2
face_id : 59
building_unit_id : null
thermal_zone_id : null
space_type_id : null
construction_set_id : null
pitched_roof_id : null
▶ daylighting_controls [0]
below_floor_plenum_height : null
floor_to_ceiling_height : null
above_ceiling_plenum_height : null
floor_offset : null
open_to_below : null
building_type_id : null
template : null
color : ■ #999933
type : space

Fig. 3. JSON Structure of space objects (visualized by JSON Viewer).

To establish spatial zones suitable for energy simulation, Floor-spaceJS defines space objects as illustrated in Fig. 3. Each space entry includes a unique identifier, a reference to a face ID, and optionally overrides story-level height settings via its own height fields. The face_id field specifies which geometric face is associated with the space. As shown in Fig. 3, space id = 3 corresponds to face 13, while space id = 39

corresponds to face 59. Additional attributes such as floor offset and visual styling may be included, although they are not essential for the core simulation process.

Taken together, this data structure facilitates a modular, multi-layered representation of building geometry. For the purpose of automation, it is essential that the algorithm accurately generates and links

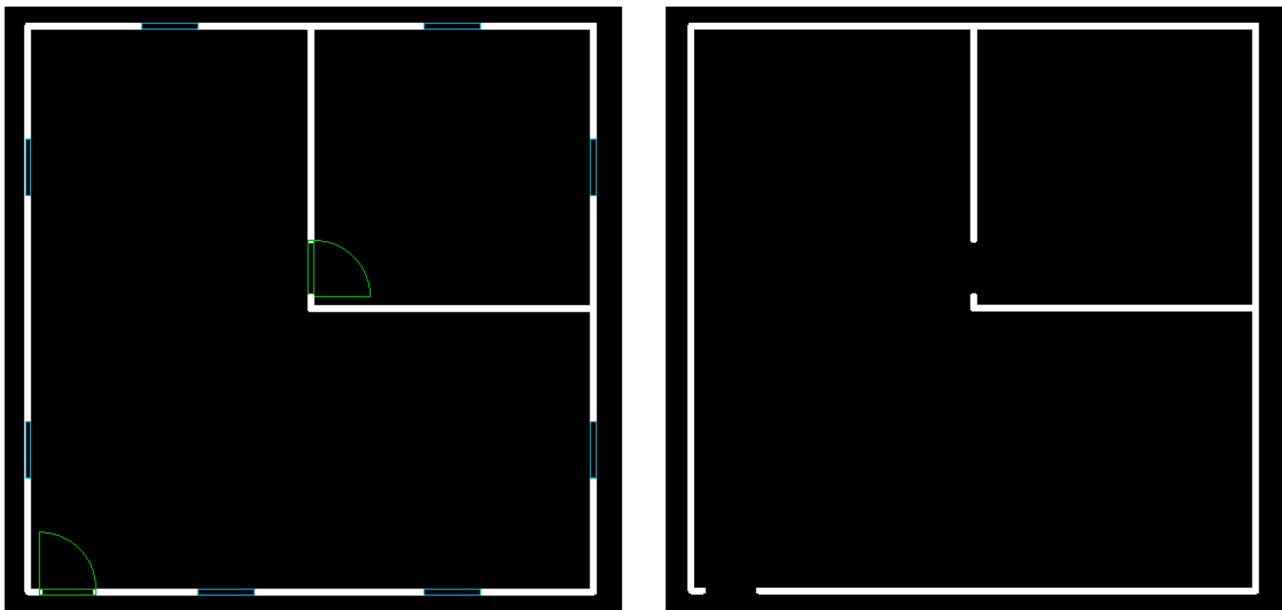


Fig. 4. Example 2D floor plan drawings: (left) with walls, doors, and windows; (right) with walls only.

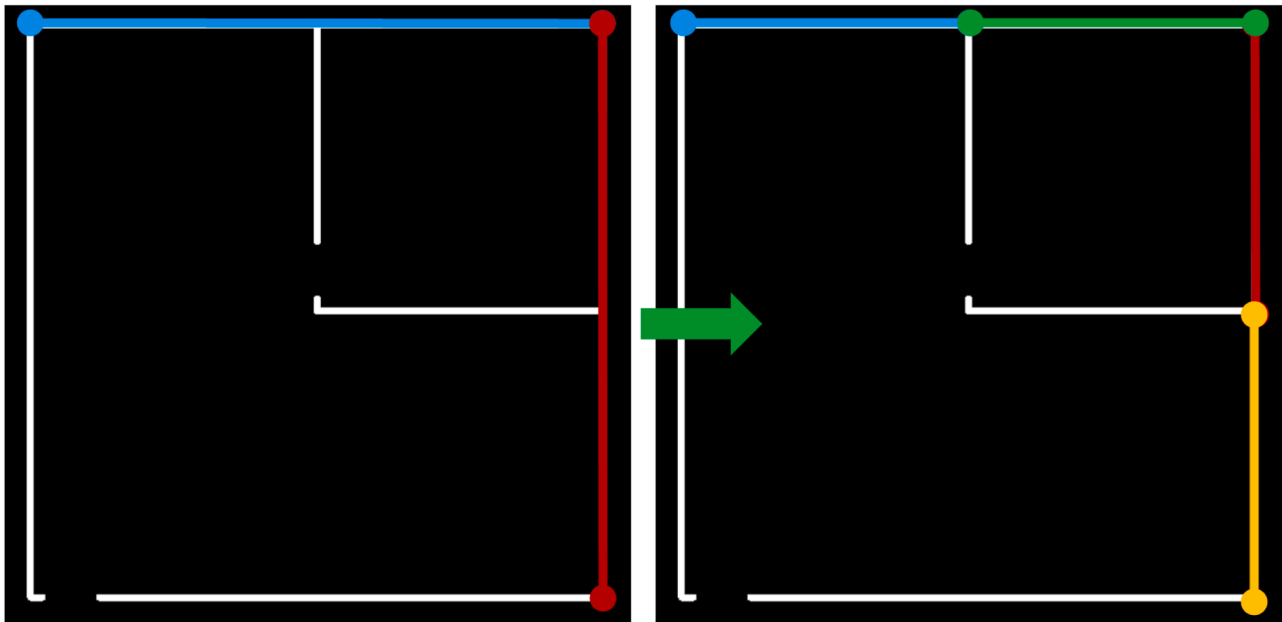


Fig. 5. The collinear line segment partitioning problem. the blue and red line segments must each be further subdivided to meet the format requirements of FloorspaceJS.

vertices, edges, and faces in a manner that satisfies FloorspaceJS's format requirements. The next section addresses how these geometric elements are extracted and synthesized from 2D CAD floor plans.

2.4. Geometric content extractable from 2D floor plan drawings

Two-dimensional CAD floor plans remain a common form of architectural documentation, especially for legacy buildings and early-stage design processes. These drawings encode substantial geometric and semantic information that can serve as a basis for building energy modeling, provided that the data can be systematically interpreted and structured.

In standard drafting practice, different building components—such as walls, doors, windows, plumbing, and annotations—are typically organized into separate layers within the CAD file. This layered structure facilitates selective data access and visual clarity. When analyzing such files for the purpose of automated space extraction, it is customary to activate only the layers relevant to spatial boundaries, most notably the wall layer. This selective activation helps isolate structural information from extraneous details.

To access and extract geometric data from DWG floor plan drawings, this study employs the pyautocad Python library. pyautocad provides an interface to AutoCAD, enabling direct interaction with drawing elements from external scripts. Through this interface, the algorithm can programmatically open a DWG file, iterate through entities in user-specified layers, and retrieve essential geometric properties such as object type, layer name, and coordinate values. This approach allows for the effective filtering of unrelated elements present in other layers.

Fig. 4 (left) depicts an example of a typical 2D CAD floor plan where the wall, door, and window layers are simultaneously enabled. While visually comprehensive, this configuration introduces visual complexity that may hinder the detection of enclosed spaces. In contrast, Fig. 4 (right) demonstrates the same floor plan with only the wall layer activated, yielding a simplified and cleaner input suitable for geometric processing.

From an algorithmic standpoint, the key geometric primitives retrievable from such simplified drawings primarily include line segments representing walls, as well as the spatial relationships implied by their connectivity. Notably, these line segments do not explicitly define

enclosed regions; instead, they must be algorithmically assembled into simple closed loops to infer the boundaries of potential thermal zones. This process requires certain preprocessing steps, such as bridging gaps in the closed curves caused by hidden door layers. These preprocessing procedures will be detailed in the following section.

2.5. The pre-processing phase of the drawing

Prior to face detection and space object generation, the raw CAD floor plan must undergo a series of preprocessing procedures to ensure geometric consistency and compatibility with the FloorspaceJS format. This stage addresses two primary challenges: incomplete wall enclosures caused by door openings and insufficient segmentation of collinear line elements.

The first issue arises from the fact that doors, typically represented as block objects in dedicated layers, interrupt the continuity of enclosing walls. This discontinuity can result in open paths that hinder the formation of valid closed loops. To resolve this, the algorithm analyzes the door layer to extract the position and orientation of each door block. Based on these attributes, supplemental wall segments are programmatically inserted to restore enclosure continuity. This reconstruction step is essential for accurate face generation in subsequent stages.

A second, more technically intricate issue pertains to the segmentation of collinear line strokes. As shown in Fig. 5, multiple walls may be drawn as a single uninterrupted line entity in CAD software such as AutoCAD. While this approach suffices for visual rendering, it conflicts with the discrete edge representation required by the FloorspaceJS schema, where each edge must be uniquely defined by a pair of vertices. To address this, the preprocessing phase employs the Bentley-Ottmann algorithm, a well-established computational geometry method for detecting all intersections among a set of line segments.

Originally proposed by Bentley and Ottmann in 1979, the algorithm operates using a plane sweep technique that significantly improves upon brute-force intersection testing [29]. It utilizes an event queue, which sorts events by x-coordinates, and a status structure, typically implemented as a balanced binary search tree, which maintains the active set of segments sorted by their vertical ordering. As the sweep line progresses, the algorithm identifies all intersection points between collinear or overlapping segments. Each identified intersection point is then used

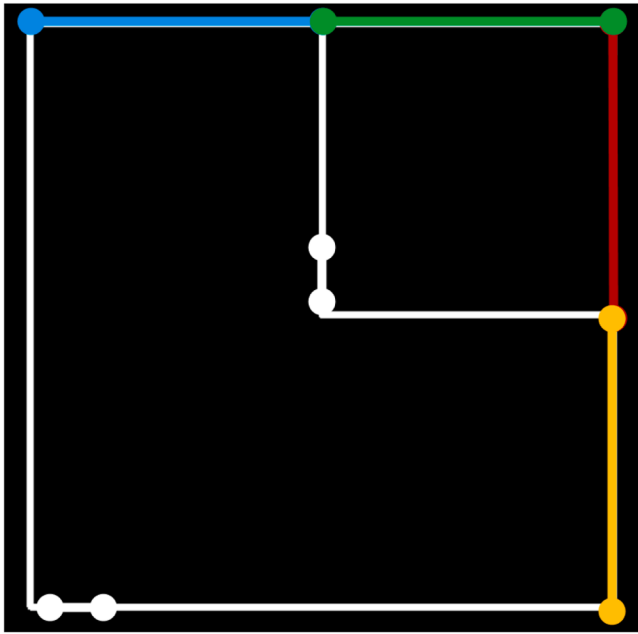


Fig. 6. The processed drawing including line segments divided at intersection points and supplementary line segments added based on the door layer.

to subdivide the corresponding line into smaller, discrete segments that conform to the FloorspaceJS format.

Once all intersections have been processed, the drawing is updated to reflect the segmented line structure. Concurrently, the previously inserted bridging segments from the door layer are merged into the updated wall layer. The resulting drawing, as illustrated in Fig. 6, features a fully segmented and enclosed wall geometry suitable for face detection.

In addition to segment subdivision, a filtering process is applied to remove freestanding walls—wall segments that are connected to the rest of the geometry at only one endpoint. These are detected by counting the frequency of each vertex occurrence across the segment list. Segments with an endpoint that appears only once are flagged as isolated and excluded from further processing. This step ensures that only structurally significant wall components are retained, enhancing both computational efficiency and geometric robustness. Upon completion of these procedures, the CAD floor plan is considered preprocessed and ready for algorithmic face detection and space object generation, as discussed in

the following section.

2.6. Face detection via path tracing algorithm

Since architectural 2D floor plans in CAD format do not explicitly define enclosed polygonal regions, an algorithmic method is required to identify closed faces suitable for simulation. To address this, we developed a path-tracing algorithm based on topological and geometric principles, primarily drawing on the Jordan Curve Theorem.

The Jordan Curve Theorem asserts that any simple closed curve in a two-dimensional plane divides the plane into two distinct and non-overlapping regions: an interior and an exterior, with the curve itself serving as the boundary [30]. In the context of this work, the term face refers to a region bounded by such a curve, constructed through a sequence of connected line segments derived from the preprocessed CAD drawing.

Since the target simulation platform, FloorspaceJS, requires face definitions in a counterclockwise direction, the algorithm prioritizes the identification of curves following this orientation. To anchor the traversal, each candidate loop is assumed to be initiated from a specially defined bottom-left point, which is the vertex with the lowest y-coordinate among all vertices on the simple closed curve. In the event of a tie, the vertex with the smallest x-coordinate among the candidates is selected. Based on the properties of such curves, it has been proven that a traversal initiated from any point on the loop can return to the origin via both clockwise and counterclockwise paths, a principle rooted in topological theory [31].

The path-tracing algorithm evaluates each vertex in the drawing as a potential starting point and constructs candidate loops under three main criteria:

1. Directional constraint:

Let the initial departure angle from the starting point be denoted as θ . To satisfy the directional constraint, θ must lie within the interval $0^\circ \leq \theta < 180^\circ$. This condition ensures that the traversal proceeds upward. It is worth noting that $\theta = 180^\circ$ is excluded because it implies the starting vertex is one of the bottom points but not the leftmost.

2. Monotonicity constraint:

During traversal, no subsequent vertex in the path may have a y-coordinate less than that of the starting vertex. Encountering such a point indicates that the starting point is not the true bottom-left point, invalidating the path for current loop detection.

3. Traversal Uniqueness via Left-Hand Rule

When multiple continuation options exist at a given vertex, the

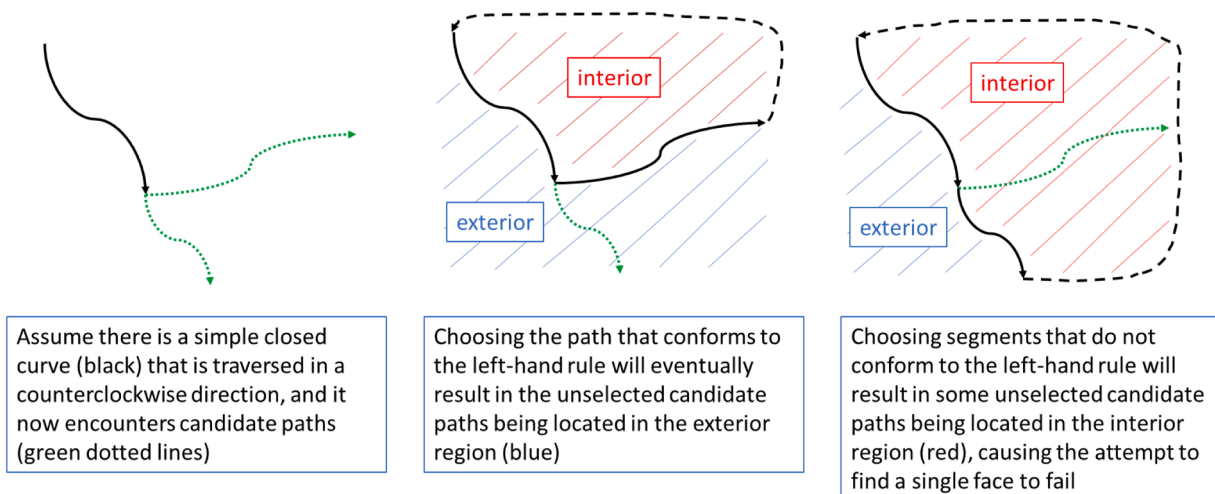


Fig. 7. Visualization of the left-hand rule for clockwise/counterclockwise path determination.

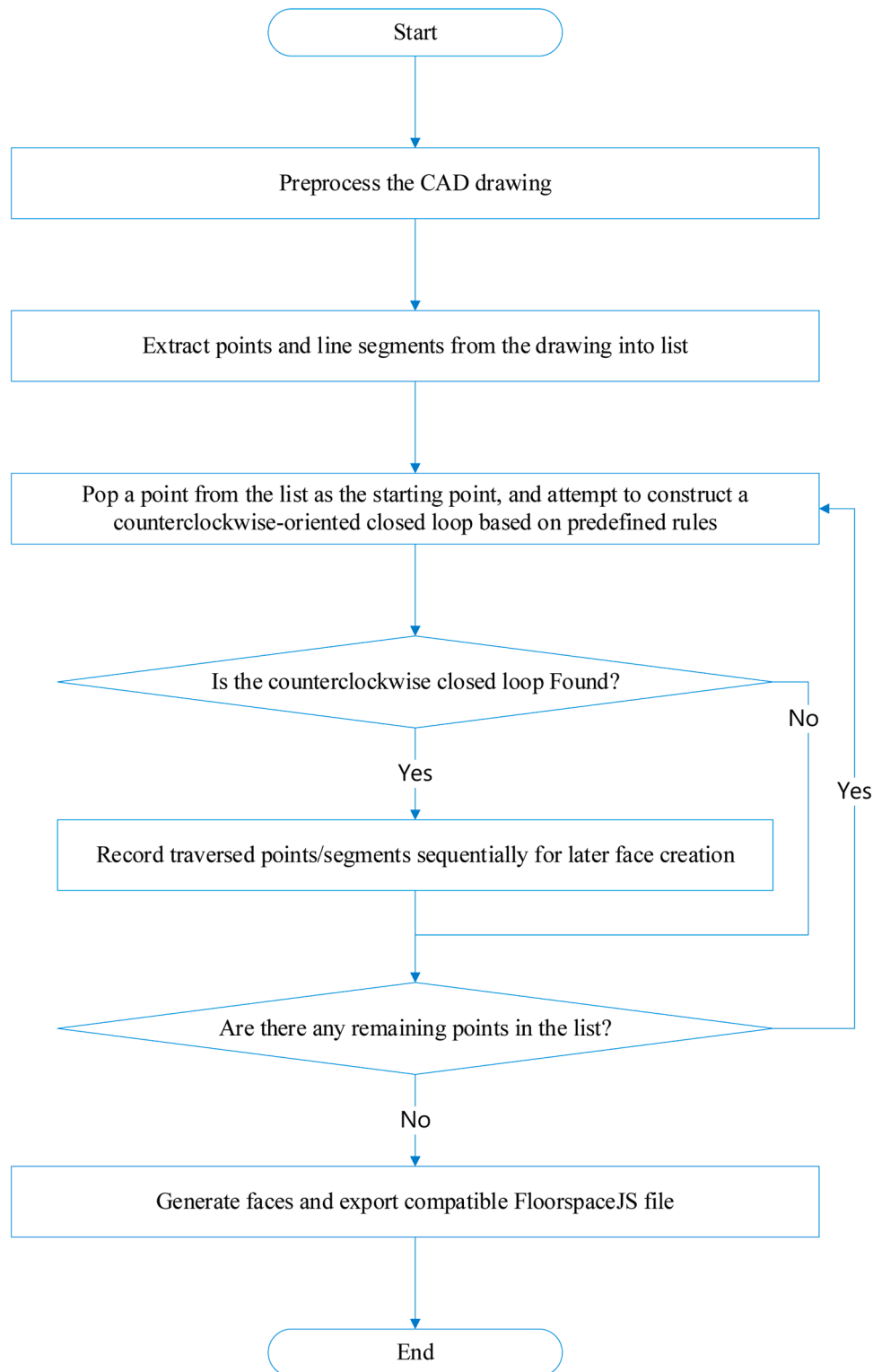


Fig. 8. The simplified flowchart of the proposed algorithm for generating FloorspaceJS-compatible objects.

algorithm applies the left-hand rule—a graph traversal heuristic derived from maze-solving strategies [32]. By keeping a virtual “left hand” on the inner boundary of the region, the traversal proceeds consistently along the outer perimeter in a counterclockwise direction. An illustration of the traversal principle is shown in Fig. 7, where the left-hand rule effectively excludes interior-crossing segments and preserves loop integrity.

The complete procedure is illustrated in Fig. 8. After the pre-processing phase, all vertices and line segments are stored in respective lists. Each vertex is examined in turn as a candidate starting point. For each valid direction, the algorithm attempts to trace a closed loop that returns to the origin, applying the above three criteria throughout traversal.

If a closed loop is successfully formed, its orientation is verified using the shoelace algorithm to confirm that it proceeds in a counterclockwise

Table 1

The results of applying the proposed algorithm to the example shown in Figure 1.

Origin Point	Starting line segment	Order of traversed line segments	Result
O			
8	11	11-57-58-X	No face found (the y-coordinate of Point 15 is less than that of Point O)
	12	N/A	No face found (Disqualified forward angle)
7	57	57-58-X	No face found (the y-coordinate of Point 15 is less than that of Point O)
	11	N/A	No face found (Disqualified forward angle)
	20	N/A	No face found (Disqualified forward angle)
56	57	N/A	No face found (Disqualified forward angle)
	58	N/A	No face found (Disqualified forward angle)
16	20	20-11-12-17-18-19	Found Face 13
	19	19-58-57-20	Found Face 59
15	18	N/A	No face found (Disqualified forward angle)
	19	N/A	No face found (Disqualified forward angle)
	58	58-57-20-19	Found Face 59
5	12	12-11-57-58-18-17	No face found (Not a counterclockwise path)
	17	17-18-19-20-11-12	Found Face 13
14	18	18-19-20-11-12-17	Found Face 13
	17	N/A	No face found (Disqualified forward angle)

direction. Valid loops are recorded and used to construct face objects, which are then mapped to corresponding space objects in the FloorspaceJS schema. Each face is uniquely defined by an ordered list of edges, and the space object is assigned metadata and visualization attributes accordingly. The process repeats iteratively until all potential bottom-left points have been evaluated and all valid faces are extracted. Duplicated faces are filtered in a post-processing step to ensure spatial uniqueness. A detailed algorithmic flowchart can be found in Fig. 10 in the Appendix.

3. Results and discussion

Table 1 summarizes the traversal path order starting from each point and the final results of whether a face was found when applying the proposed algorithm to the example shown in Fig. 1. It can be observed that each face was found more than once, demonstrating the feasibility of the proposed algorithm and the necessity of removing duplicate face and space objects. On the other hand, it can also be observed that defining the bottom-left point and checking the starting forward angle effectively reduces the computational load. In the example shown in Table 1, 50 % of the starting paths could be immediately excluded, thereby accelerating the computation process.

To further evaluate the algorithm's applicability in real-world scenarios, an actual floor plan of a residential unit in Taipei, Taiwan (Fig. 9, left) was applied. The unit, approximately 100 square meters, includes three bedrooms, a bathroom, and an open kitchen-living area. The algorithm successfully detected the majority of enclosed spaces and generated a valid FloorspaceJS-compatible JSON output (Fig. 9, right).

However, one notable limitation arose in the open-plan region labeled Space 1. This space spans the living room, dining area, and corridor, which lack explicit internal partitions in the source CAD drawing. As a result, the algorithm identified the entire zone as a single unified space. This behavior is expected given the algorithm's reliance

on geometric enclosure only. Unlike prior approaches such as Domínguez et al. [13], which integrate geometric analysis with block-based semantic cues—such as user-defined opening blocks and layer-specific information—our method deliberately avoids reliance on such annotations. This design choice minimizes input assumptions and enhances automation, but introduces ambiguity when room boundaries are implied rather than explicitly drawn.

When compared to early methods like Lewis and Séquin [14], which reconstruct 3D building models from 2D plans, our method takes a different direction by focusing on simulation readiness rather than full geometric fidelity. Their work emphasizes consistency in volumetric reconstruction, whereas ours prioritizes efficient conversion to FloorspaceJS to support energy modeling workflows. This distinction is especially relevant for retrofit or early-stage projects where full 3D modeling is unnecessary or impractical.

Additionally, BIM-based methods such as those discussed in the introduction section rely on rich IFC structures and structured spatial data to perform zone detection or indoor navigation modeling. While these approaches offer high accuracy and semantic completeness, they are inherently dependent on well-maintained BIM environments. By contrast, our method is tailored for CAD-based inputs—still widely used in industry—and does not require BIM integration, making it well-suited for legacy projects and firms with limited BIM adoption.

Clearly, if the segments in the drawing are not properly categorized using layers, a disorganized drawing will prevent the proposed algorithm from correctly identifying which segments represent walls that separate spaces, leading to incorrect results. Fortunately, for ease of future reference, the vast majority of engineering and architectural teams properly apply layers when classifying drawing elements. Therefore, by simply masking the unnecessary layers, the required wall segments can be easily obtained. It should be noted that drawing practices may vary among teams, and in cases where preprocessing is insufficient, manual creation of simplified floor plans may be necessary. It is also important to note that creating a simple floor plan on a new layer in a CAD drawing is far more efficient than doing the same task in building energy modeling software. The former allows for the quick reproduction of precise space dimensions using object snap features, while the latter requires engineers to laboriously input space dimensions or trace the contour of a reference image, which compromises dimensional accuracy.

It is worth noting that the results of this study indicate that there is still a long way to go before achieving the ideal of fully automated building energy simulation. For instance, the JSON format generated in this study only contains information about spaces. To perform a simulation, additional steps would be necessary, such as adding doors and windows, assigning space types to each space, and grouping spaces into thermal zones. Moreover, some spaces may have varying ceiling heights that require special adjustments. Despite these challenges, the approach proposed in this study allows building energy modeling engineers to save a significant amount of time by eliminating the need to repetitively interpret drawings and input dimensions for each space. This enables them to focus on handling exceptional cases, such as the aforementioned issue of uneven ceiling heights.

On the other hand, some of these tasks may have further potential for automation under the right conditions. For example, in the automated setting of windows, the installation location and width can typically be obtained from the floor plan drawing, but the window height and sill height must be derived from the elevation drawing. If a method could be developed to establish the correspondence between the floor plan and the elevation drawing, it would be entirely feasible to create FloorspaceJS-compatible objects and inject them into the JSON file. However, such a project would involve collaboration across multiple drawing types and the application of various drawing recognition techniques, which warrants further research in the future.

Regarding the space information (such as the actual name of the space and its space type, e.g., "Conference Room 101" categorized as a



Fig. 9. An actual floor plan CAD drawing with only wall and door layers activated (left) and the generated FloorspaceJS-compatible JSON file (right).

meeting room), it currently needs to be manually input during the post-processing phase. In terms of energy simulation accuracy, the space name may not have a significant impact. The more critical factor is the space type, as different space types determine various parameters such as occupant density, lighting power density, and plug load power density. Some specific spaces, like gymnasiums, may even require special thermostat settings to provide a thermally comfortable environment for the occupants.

Fortunately, in typical floor plan drawings, it is common practice to place a text label within a space to describe its information, such as the name, number, or even the area. In the future, it may be possible to determine the coordinates of a text label and identify which face these coordinates fall within, and then use an AI model for semantic analysis to parse the label's text and determine the space type. Once this value is established, the corresponding space type templates from the built-in library in OpenStudio can be used to quickly complete this part of the setup. After the spaces are configured, automatically grouping them into

thermal zones may involve overlaying space drawings with HVAC system diagrams and using the contours of both to determine the thermal zone boundaries. Clearly, such a complex problem is better suited for resolution using computer vision techniques.

The proposed algorithm operates with a time complexity of $O(NM)$, where N represents the total number of vertices in the drawing, and M corresponds to the total number of line segments. This complexity arises because each vertex must be examined individually, and the examination process involves querying all line segments. Despite this, the number of spaces on a typical single floor rarely exceeds 500, and the number of line segments representing walls generally does not surpass 10,000. Given the current state of computational power, even the largest and most complex floor plans are likely to be processed within seconds.

In summary, the results indicate that the proposed method offers a pragmatic trade-off between simplicity and functionality. It provides substantial time savings in space object generation and supports integration with simulation tools such as OpenStudio. Nevertheless, certain

limitations remain, which are discussed in the following section.

4. Limitations and assumptions

While the proposed algorithm provides an efficient means to automate space object generation from 2D CAD drawings, several limitations and assumptions must be acknowledged. For example, the algorithm only supports straight-line segments. Curved walls represented by splines or arcs in CAD drawings must be manually converted to segmented polylines before processing. It is worth noting that even traditional manual modeling requires this step, since most mainstream BEM software do not support curved lines as valid geometric objects.

Since the current algorithm is based on 2D CAD drawings that lack 3D information, it can only generate models with uniform height across the entire level. It is unable to handle cases involving misaligned floor or ceiling elevations or sloped roofs. A common example is internal atriums, which may be incorrectly treated as interior spaces by the current algorithm. These exceptional conditions must be manually adjusted by engineers after the automated modeling process is completed.

The current algorithm also lacks the reasoning capability to detect inconsistencies or errors in the CAD drawing, it must assume that the drawing is entirely accurate and rely solely on the information provided. If there are any mistakes in the drawing, the resulting space recognition may also be incorrect. It is worth noting that the current algorithm possesses a certain level of error-handling capability. For instance, if duplicate walls exist in the drawing, they may lead to the detection of duplicate spaces. However, since the algorithm inherently has a chance of detecting redundant spaces even under normal conditions, a process is applied in the final stage to eliminate such redundancies, thereby preventing any issues.

In contrast, disconnected wall segments present a more complex situation. If the wall is not structurally necessary—such as an unintended element or drafting artifact—it will be identified as a free-standing wall and automatically removed by the algorithm without causing errors. However, if the disconnected wall is supposed to exist and enclose a space, the algorithm will fail to recognize the intended geometry. In such cases, manual intervention is required by the engineer to add connecting segments and complete the enclosure.

5. Conclusion

In this paper, an algorithm was proposed to reduce the lengthy process of creating building geometries required for Building Energy Simulation by leveraging existing 2D CAD floor plan drawings. The proposed algorithm begins with a preprocessing step that enhances raw CAD data. It then detects closed regions (faces) using a path traversal algorithm informed by the Jordan Curve Theorem and the left-hand

rule. These faces are transformed into the geometric and space objects required by the FloorspaceJS format, thereby enabling their direct use in OpenStudio for building energy simulations.

The algorithm has a time complexity of $O(NM)$, where N is the total number of vertices and M is the total number of line segments in the relevant layer of drawing. Given that N and M are typically within manageable limits, the algorithm is expected to complete its task in seconds. While the proposed approach significantly reduces manual effort, several limitations must be acknowledged. It assumes the 2D CAD input is free of major errors and supports only line-based geometry—curved elements, 3D elevation changes, and ambiguous spatial boundaries require manual intervention.

For future work, we plan to incorporate more real-world floor plans to evaluate the robustness of the algorithm across diverse building types. Furthermore, we intend to explore the integration of elevation data from section drawings, and the use of natural language processing to automatically assign space types from CAD text labels. These enhancements will further reduce manual post-processing and increase modeling fidelity.

In summary, there is no doubt that when BIM data is available, directly converting it into an OpenStudio-compatible file format is a faster and more efficient approach. However, compared with element-based BIM methods, the algorithm proposed in this study offers a lightweight alternative. It serves as a practical solution for accelerating the modeling process in cases where BIM data is unavailable. Such situations, where only CAD drawings are accessible, are still common in many architectural design practices today.

CRediT authorship contribution statement

Wei-Jen Chen: Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization. **Min-Cheng Tu:** Writing – review & editing, Writing – original draft, Validation, Data curation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

The author would like to thank the National Science and Technology Council of the Republic of China (Taiwan) for their financial support of this research under Contract no. NSTC 112-2221-E-003-004-MY2

Appendix: The detailed flowchart of the proposed algorithm for generating FloorspaceJS-compatible objects

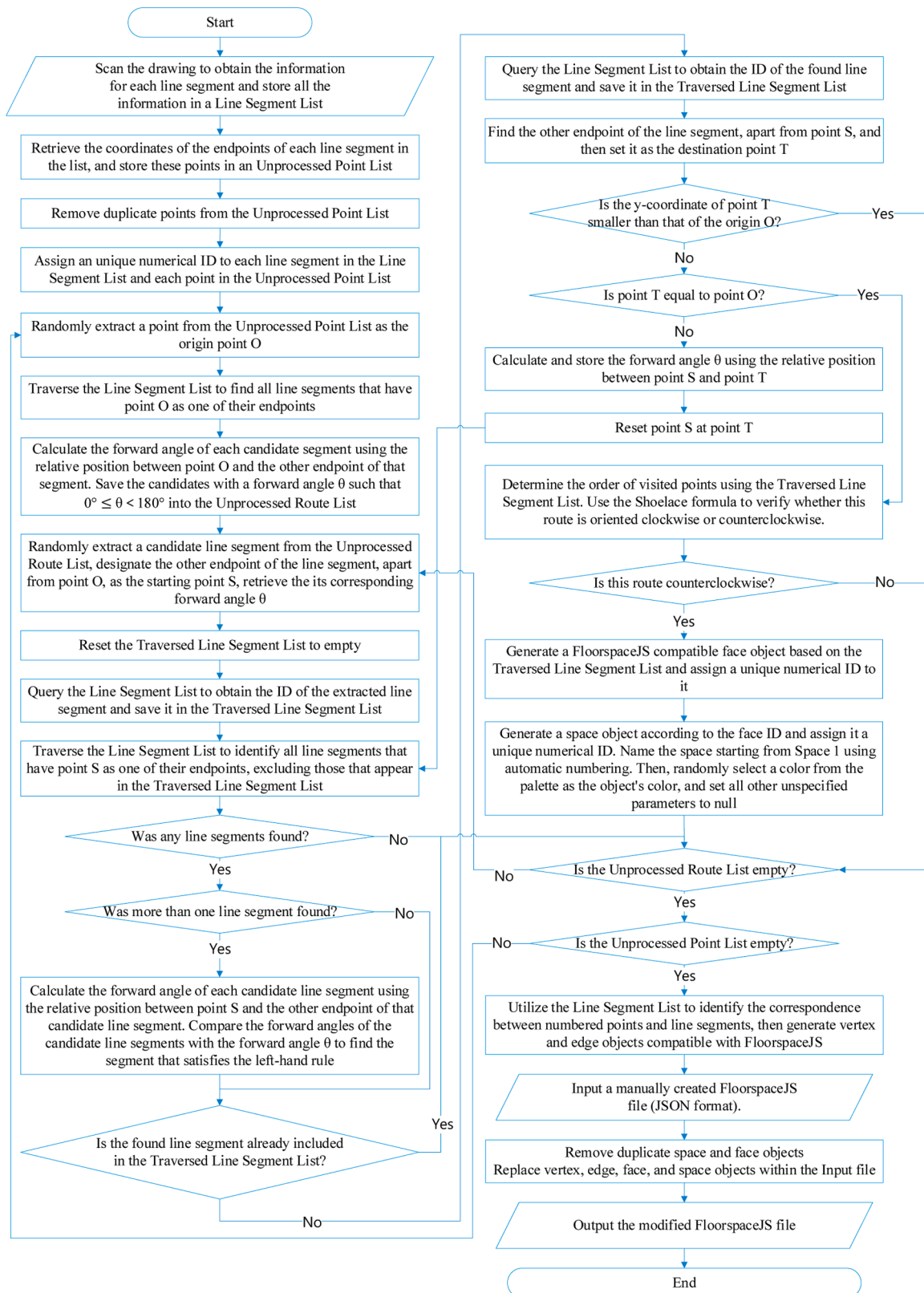


Fig. 10. The detailed flowchart of the proposed algorithm for generating FloorspaceJS-compatible objects.

The entire procedure begins by scanning the wall layers within the pre-processed floorplan CAD drawing, obtaining all the edge information, and storing it in the Line Segment List. Subsequently, the algorithm uses this edge information to extract and record all endpoints, which are then stored in the Unprocessed Point List, awaiting future examination.

The examination process involves extracting a point from the Unprocessed Point List to serve as the origin point O . The Line Segment List is then

queried to find all segments with this point as one of their endpoints that meet the specified criteria (such as the departure angle mentioned in the previous section). These segments are stored in the Unprocessed Route List as candidate starting paths. From the Unprocessed Route List, one path is extracted, and the other endpoint, which is not point O, is set as the starting point S. The Line Segment List is scanned again to find all segments that have point S as one of their endpoints. If more than one segment is found, the left-hand rule is applied to determine the appropriate path to select. Once a path is chosen, the algorithm identifies another target point T through this path, which is then subjected to two conditions: (1) whether the y-coordinate is less than that of the origin O (if so, it does not meet the aforementioned second criterion), and (2) whether it equals the origin O (if so, the closed loop is complete). If both conditions are negative, it indicates that further computation is necessary. The traversed segment is then recorded in the Traversed Line Segment List, point T is reset as point S, and the above path-searching process is repeated. This continues until the loop either returns to the origin O or no further paths are available.

When point T equals the origin O, the shoelace algorithm is used to verify whether the path was completed in a counterclockwise direction. If so, according to the aforementioned third criterion, it forms a simple closed curve with no other segments inside, enclosing the face that the algorithm is designed to find. At this point, the Traversed Line Segment List is queried, and the face object is output. This face object is then used to generate a space object, which is automatically assigned a unique numerical ID and a randomly selected color from the palette listed in Table 2 as its designated color. This loop continues until the Unprocessed Point List is emptied, indicating that all endpoints have been examined.

Finally, after removing duplicate space and face objects, a manually created FloorspaceJS template is input, and the algorithm-generated vertex, edge, face, and space objects replace the template's contents. The modified file is then output, marking the conclusion of the process.

Table 2

The palette utilized to define the color of the space object.

Color Name	Hex Code
Green	#00FF00
Blue	#0000FF
Yellow	#FFFF00
Brown	#A52A2A
Orange	#FFA500
Purple	#800080
Pink	#FFC0CB
Black	#000,000
Gray	#808,080
Gold	#FFD700

Data availability

Data will be made available on request.

References

- [1] T. Han, Q. Huang, A. Zhang, Q. Zhang, Simulation-based decision support tools in the early design stages of a green building-A review, *Sustain* (2018), <https://doi.org/10.3390/su10103696>.
- [2] M.N. Hamedani, R.E. Smith, Evaluation of performance modelling: optimizing simulation tools to stages of architectural design, in: *Procedia Eng.* 2015, <https://doi.org/10.1016/j.proeng.2015.08.513>.
- [3] M. Nik-Bakht, R.O. Panizza, P. Hudon, P.Y. Chassain, M. Bashari, Economy-energy trade off automation – A decision support system for building design development, *J. Build. Eng.* (2020), <https://doi.org/10.1016/j.jobe.2020.101222>.
- [4] ASHRAE, ANSI/ASHRAE Standard 140-2001: Standard Method of Test for the Evaluation of Building Energy Analysis Computer Programs., Ashrae (2014).
- [5] V. Shabunko, C.M. Lim, S. Mathew, EnergyPlus models for the benchmarking of residential buildings in Brunei Darussalam, *Energy Build.* (2018), <https://doi.org/10.1016/j.enbuild.2016.03.039>.
- [6] A.Parker L.Brackney, K.Benne D.Macumber, The OpenStudio software development Kit, in: *Build. Energy Model. with OpenStudio* (2018), https://doi.org/10.1007/978-3-319-77809-9_9.
- [7] A. Ciccozzi, T. deRubeis, D. Paoletti, D. Ambrosini, BIM to BEM for Building Energy Analysis: a review of interoperability strategies, *energies*. (2023). <https://doi.org/10.3390/en16237845>.
- [8] V. Bazjanac, L. Berkeley, Space boundary requirements for modeling of building geometry for energy and other performance simulation, in: *Proc. CIB W78 2010 27th Int. Conf.* 2010.
- [9] C. vanTreeck, E. Rank, Dimensional reduction of 3D building models using graph theory and its application in building energy simulation, *Eng. Comput.* (2007), <https://doi.org/10.1007/s00366-006-0053-7>.
- [10] G.N. Lillis, G.I. Giannakis, D.V. Rovas, Automatic generation of second-level space boundary topology from IFC geometry inputs, *Autom. Constr.* (2017), <https://doi.org/10.1016/j.autcon.2016.08.044>.
- [11] N.L. Jones, C.J. McCrone, B.J. Walter, K.B. Pratt, D.P. Greenberg, Automated translation and thermal zoning of digital building models for energy analysis, in: *Proc. BS 2013 13th Conf. Int. Build. Perform. Simul. Assoc.* 2013.
- [12] C.M. Rose, V. Bazjanac, An algorithm to generate space boundaries for building energy simulation, *Eng. Comput.* (2015), <https://doi.org/10.1007/s00366-013-0347-5>.
- [13] Á.L.García B.Domínguez, F.R. Feito, Semiautomatic detection of floor topology from CAD architectural drawings, *CAD Comput. Aided Des.* (2012), <https://doi.org/10.1016/j.cad.2011.12.009>.
- [14] R. Lewis, C. Séquin, Generation of 3D building models from 2D architectural plans, *CAD Comput. Aided Des.* (1998), [https://doi.org/10.1016/S0010-4485\(98\)00031-1](https://doi.org/10.1016/S0010-4485(98)00031-1).
- [15] G.N. Lillis, K. Katsigarakis, D.V. Rovas, Automatic IFC data enrichment with space geometries for building energy performance simulations, in: *Build. Simul. Conf. Proc.* 2022, <https://doi.org/10.26868/25222708.2021.30888>.
- [16] W.Y. Lin, P.H. Lin, H.P. Tserng, Automating the generation of indoor space topology for 3D route planning using BIM and 3D-GIS techniques, in: *ISARC 2017 - Proc. 34th Int. Symp. Autom. Robot. Constr.* 2017, <https://doi.org/10.22260/isarc2017/0060>.
- [17] Y. Fang, S.-A. Mitoulis, D. Boddice, J. Yu, J. Ninic, Scan-to-BIM-to-sim: automated reconstruction of digital and simulation models from point clouds with applications on bridges, *Results Eng.* 25 (2025) 104289, <https://doi.org/10.1016/j.rineng.2025.104289>.
- [18] M. Altaf, R. Jaffari, W.S. Alalaoul, M.A. Musarat, S. Ammad, Developing automated strategy of life cycle cost analysis (LCCA) with building information modeling (BIM) integration for building projects, *Results Eng.* 25 (2025) 104179, <https://doi.org/10.1016/j.rineng.2025.104179>.
- [19] X. Lebeugue, J.K. Aggarwal, Automatic creation of architectural CAD models, in: *Proc. IEEE CAD-Based Vis. Work.* 1994, <https://doi.org/10.1109/cadvis.1994.284513>.
- [20] B. Bortoluzzi, I. Efremov, C. Medina, D. Sobieraj, J.J. McArthur, Automating the creation of building information models for existing buildings, *Autom. Constr.* (2019), <https://doi.org/10.1016/j.autcon.2019.102838>.
- [21] V.A. Cicirello, W.C. Regli, A flexible and extensible approach to automated CAD/CAM format classification, *Comput. Graph.* (2013), <https://doi.org/10.1016/j.cag.2013.03.007>.
- [22] A. Tavakoli, K.L. Klika, Construction management with AutoCAD, *J. Manag. Eng.* (1991), [https://doi.org/10.1061/\(asce\)9742-597x\(1991\)7:3\(267\)](https://doi.org/10.1061/(asce)9742-597x(1991)7:3(267)).
- [23] D. Zhao, C.W. Pan, H.Y. Li, Resolution of DWG files based on dwgdirect and 3D modeling, in: *Appl. Mech. Mater* (2013), <https://doi.org/10.4028/www.scientific.net/AMM.246-247.385>.
- [24] Y. Zhang, X. Shi, Development of AutoCAD graphic database management system based on component technology, in: *Adv. Mater. Res.* 2014, <https://doi.org/10.4028/www.scientific.net/AMR.998-999.1394>.
- [25] Q. Wang, Q. Wu, The design and application of automatic detection and correction system on spatial conflict to line object based on DWGDirect, in: *2010 2nd Conf. Environ. Sci. Inf. Appl. Technol. ESIAT*, 2010, p. 2010, <https://doi.org/10.1109/ESIAT.2010.5567422>.

- [26] H.Y. Li, Y. Chen, D. Zhao, The design and implementation of 3D visualization pipeline information system based on 2D figures, in: *Appl. Mech. Mater.*, 2013, <https://doi.org/10.4028/www.scientific.net/AMM.246-247.1241>.
- [27] S.Mehendarge P.Shende, V.Kasturkar A.Goel, M.Takalikar S.Chougule, Automating and optimizing the design process of solar module installation and providing necessary forecasts using machine learning algorithm, in: *ACM Int. Conf. Proceeding Ser.*, 2018, <https://doi.org/10.1145/3195106.3195165>.
- [28] J. Juninawan, Y.R. Alkhaly, R. Mirsa, Implementation of building component drawing automation to improve time efficiency and accuracy in the construction planning process, *Int. J. Eng. Bus. Soc. Sci.* (2023), <https://doi.org/10.58451/ijebss.v1i06.98>.
- [29] T.A.Ottmann J.L.Bentley, Algorithms for reporting and counting geometric intersections, *IEEE Trans. Comput.* (1979), <https://doi.org/10.1109/TC.1979.1675432>.
- [30] X. Zhang, A proof of the Jordan curve theorem, *Nat. Sci.* (2019), <https://doi.org/10.4236/ns.2019.1112037>.
- [31] T.C. Hales, The Jordan curve theorem, formally and informally, *Am. Math. Mon.* (2007), <https://doi.org/10.1080/00029890.2007.11920481>.
- [32] A. BakarSayutiSaman, I. Abdramane, Solving a reconfigurable maze using hybrid wall follower algorithm, *Int. J. Comput. Appl.* (2013), <https://doi.org/10.5120/14097-2114>.